🐍 Clase 3: Programación en Python II

Objetivo de aprendizaje

Al finalizar la clase, los estudiantes podrán:

1. Utilizar listas, tuplas y diccionarios para organizar información.
2. Combinar ciclos y condicionales para procesar datos.
3. Crear funciones con parámetros y valores de retorno.
4. Resolver problemas sencillos mediante programación.
5. Leer, probar y modificar código generado por IA.

9:00–9:20 | 1. Recuperando lo aprendido

Objetivo: Activar conocimientos previos antes de introducir conceptos nuevos.

No empezaría con diapositivas. Abriría directamente Jupyter Notebook.

Actividad: ¿Qué hace este código?

```
numeros = [3, 7, 10, 15, 20]
for numero in numeros:
        if numero > 10:
                print(numero)
```

Antes de ejecutarlo, pregúntales:

- ¿Qué tipo de dato es `numeros`?
- ¿Qué hace el `for`?
- ¿Qué hace el `if`?
- ¿Cuántas veces se ejecuta el ciclo?
- ¿Qué valores se van a imprimir?

Después ejecútalo.

Resultado:

15
20

Ahora modifica la condición:

if numero >= 10:

Y pregúntales qué cambia.

Segunda actividad: detectar un error

```
edad = "25"

if edad >= 18:
        print("Mayor de edad")
```

¿Por qué no funciona?

Aquí recuperas los tipos de datos y la conversión con `int()`.

Mensaje pedagógico: antes de escribir código nuevo, necesitamos aprender a leer y entender el que ya existe.

9:20–9:45 | 2. Estructuras de datos

Objetivo: Comprender cómo almacenar y organizar colecciones de información.

La clase pasada ya utilizaron listas. Ahora ampliamos el concepto.

Listas

```
calificaciones = [85, 90, 75, 100]
```

Explícales que son colecciones ordenadas y modificables.

Introduce dos operaciones nuevas:

```
calificaciones.append(95)
len(calificaciones)
```

Y aprovecha para mostrar cómo acceder a elementos:

```
calificaciones[0]
```

Aquí vale la pena recordar que Python comienza a contar desde cero.

Tuplas

```
coordenadas = (20.67, -103.35)
```

Explícales que se parecen a las listas, pero no podemos modificar sus elementos.

Una comparación sencilla:

```
lista = [1, 2, 3]
```

```
tupla = (1, 2, 3)
```

No necesitan dominar todas sus diferencias. Basta con entender cuándo están trabajando con cada estructura.

Diccionarios

```
estudiante = {    "nombre": "Ana",    "edad": 25,    "calificacion": 95}
```

Muéstrales:

```
estudiante["nombre"]
```

Y después:

```
estudiante["calificacion"] = 100
```

Aquí puedes conectar con Ciencia de Datos:

> "Muchas veces los datos no son solamente números. Tenemos información relacionada: nombres, edades, ciudades, calificaciones. Necesitamos estructuras que nos permitan organizarla."

Mini ejercicio

Pídeles crear un diccionario que contenga:

- Su nombre.
- Su edad.
- Su comida favorita.
- Si les gusta Python (`True` o `False`).

Después deben imprimir únicamente su comida favorita.

No profundizaría más en diccionarios por ahora.

9:45–10:10 | 3. Procesamiento de datos con ciclos

Esta sección es importantísima porque prepara la lógica que utilizarán dentro de las funciones.

Concepto 1: acumuladores

Empieza con una lista de ventas.

```
ventas = [100, 200, 300, 400]
total = 0
for venta in ventas:
        total = total + venta
print(total)
```

Resultado:

1000

Detente en esta línea:

```
total = total + venta
```

Explícales que una variable puede actualizar su valor en cada iteración.

Yo incluso haría una tablita en vivo:

Iteración	Venta	Total
1	100	100
2	200	300
3	300	600
4	400	1000

Esto les ayuda a visualizar lo que sucede dentro del ciclo.

Concepto 2: construir una nueva lista

```
calificaciones = [50, 80, 95, 60, 75]
aprobados = []
for calificacion in calificaciones:
        if calificacion >= 70:
                aprobados.append(calificacion)

print(aprobados)
```

Resultado:

[80, 95, 75]

Aquí introduces formalmente el patrón:

Recorrer → evaluar → guardar.

Este patrón es fundamental para el trabajo posterior con datos.

Mini reto

Modificar el código para obtener únicamente las calificaciones menores a 70.

No les des la respuesta inmediatamente.

10:10–10:40 | 4. Introducción a funciones

Aquí empieza el tema central del día. 💜

Primero: ¿qué problema resuelven las funciones?

Escribe:

```
print("Hola Ana")
print("Hola Pedro")
print("Hola Luis")
```

Y pregúntales:

¿Qué pasaría si tuviéramos que saludar a 1,000 personas?

Después:

```
def saludar(nombre):
    print("Hola " + nombre)
```

Ejecuta:

```
saludar("Ana")
saludar("Pedro")
saludar("Luis")
```

Explícales las partes:

```
def saludar(nombre):
    print("Hola " + nombre)
```

- `def`: indica que estamos definiendo una función.
- `saludar`: nombre de la función.
- `nombre`: parámetro que recibe.
- `:`: indica el inicio del bloque.
- Indentación: identifica las instrucciones que pertenecen a la función.

Parámetros y argumentos

Aquí introduciría la diferencia:

Parámetro: variable definida en la función.

Argumento: valor que proporcionamos cuando llamamos a la función.

Ejemplo:

```
def saludar(nombre):
    print("Hola " + nombre)

saludar("Hayde")
```

`nombre` es el parámetro.

`"Hayde"` es el argumento.

Segundo ejemplo

```
def calcular_area(base, altura):
    area = base * altura
    print(area)
```

Ejecuta:

```
calcular_area(5, 10)
```

Después cambia los valores.

Esto les ayuda a entender que una función puede reutilizarse con diferentes entradas.

10:40–11:00 | 5. Print vs. Return

Esta sería una de las secciones a las que más atención prestaría.

Primero:

```
def sumar(a, b):
    print(a + b)
```

Ejecuta:

```
resultado = sumar(5, 3)
print(resultado)
```

Obtendrán:

8
None

¿Por qué?

Porque la función imprimió el resultado, pero no devolvió ningún valor.

Ahora:

```
def sumar(a, b):
        return a + b
```

Ejecuta:

```
resultado = sumar(5, 3)
print(resultado)
```

Resultado:

8

Y ahora:

```
resultado * 2
```

Resultado:

16

Concepto fundamental

`print()` muestra información.

`return` devuelve información para que pueda utilizarse después.

Les explicaría que una función puede recibir información, procesarla y devolver un resultado.

Ese es el modelo mental que quiero que se lleven:

ENTRADA → PROCESAMIENTO → SALIDA

Y es también el modelo que utilizarán en los ejercicios de programación del examen.

11:00–11:15 | Descanso ☕

11:15–11:45 | 6. Construyendo nuestra primera función con lógica

Ahora combinamos lo que aprendieron.

Ejercicio: Clasificador de calificaciones

Instrucciones:

Crear una función que reciba una calificación y devuelva:

- `"Excelente"` si es mayor o igual a 90.
- `"Aprobado"` si está entre 70 y 89.
- `"Reprobado"` si es menor a 70.

Primero pídeles que describan la lógica.

Después construyen:

```
def clasificar_calificacion(calificacion):
	if calificacion >= 90:
		return "Excelente"
	elif calificacion >= 70:
		return "Aprobado"
	else:
		return "Reprobado"
```

Ejecutan:

```
clasificar_calificacion(95)
clasificar_calificacion(75)
clasificar_calificacion(50)
```

Ahora el reto

Pídeles modificar la función para agregar una nueva categoría:

"Perfecto" cuando la calificación sea exactamente 100.

Aquí quiero que aprendan algo importante: el orden de las condiciones importa.

11:45–12:15 | 7. Funciones que procesan listas

Aquí ya entramos en problemas más cercanos a Ciencia de Datos.

Ejercicio: Filtrar calificaciones

Instrucciones:

Crear una función que reciba una lista de calificaciones y devuelva solamente las aprobatorias, se aprueba con 70.

Primero construyan la lógica entre todos.

Después:

```
def obtener_aprobados(calificaciones):
        aprobados = []
        for calificacion in calificaciones:
                if calificacion >= 70:
                        aprobados.append(calificacion)
        return aprobados
```

Ejecutan:

```
calificaciones = [50, 80, 95, 60, 75]
obtener_aprobados(calificaciones)
```

Resultado:

[80, 95, 75]

Ahora deben modificarla

Reto 1: Que devuelva solamente las calificaciones reprobatorias.

Reto 2: Que reciba también la calificación mínima aprobatoria como parámetro.

Por ejemplo:

```
obtener_aprobados(calificaciones, 80)
```

Aquí ya estamos practicando funciones con múltiples parámetros.

Y estamos desarrollando una habilidad importante: **generalizar soluciones**.

12:15–12:40 | 8. Funciones con acumuladores

Ejercicio: Total de ventas

Instrucciones:

Crear una función que reciba una lista de ventas y devuelva la suma total.

Primero, que intenten escribirla sin ayuda durante unos minutos.

Después revisan:

```
def calcular_total(ventas):
    total = 0
    for venta in ventas:
        total = total + venta
    return total
```

Ejecutan:

```
calcular_total([100, 200, 300])
```

Resultado:

600

Reto adicional

Modificar la función para que devuelva el promedio de las ventas.

Aquí necesitarán:

len(ventas)

Y podrán reutilizar la lógica anterior.

Para este primer ejercicio, asuman que la lista tiene al menos una venta.

12:40–12:55 | 9. Python + IA: aprender a validar código

Aquí retomaría lo que les contaste la semana pasada sobre vibe coding.

Pero en lugar de explicar nuevamente el concepto, lo llevaría a la práctica.

Actividad: ¿La IA hizo bien su trabajo?

Muéstrales este código:

```
def obtener_mayores(numeros):
    mayores = []
    for numero in numeros:
        if numero < 10:
            mayores.append(numero)
    return mayores
```

Les dices:

> "Le pedí a una IA que me generara una función para obtener los números mayores a 10 y me entregó esto."

Preguntas:

¿Está bien? ¿Qué hace realmente?

Después pídeles corregirlo.

Y finalmente probarlo con:

obtener_mayores([5, 10, 15, 20, 25])

Resultado esperado:

[15, 20, 25]

Mensaje de cierre

> "La IA puede ayudarnos a escribir código, pero necesitamos entender el problema, revisar la solución y comprobar que realmente hace lo que esperamos."

Este ejercicio conecta directamente con una de las habilidades que evaluaremos en el nuevo examen.

12:55–13:00 | 10. Cierre y preparación para el examen

Yo terminaría con cinco preguntas rápidas:

1. ¿Qué diferencia hay entre una lista y una tupla?
2. ¿Para qué sirve `append()`?
3. ¿Qué diferencia hay entre `print()` y `return`?
4. ¿Qué sucede cuando combinamos un `for` con un `if`?
5. ¿Cómo podemos comprobar que una función funciona correctamente?

Y les diría:

> "La próxima semana tendrán su primer examen. No espero que memoricen todos los comandos de Python. Espero que puedan entender un problema, pensar cómo resolverlo, construir una solución y comprobar que funciona. Pueden apoyarse en sus apuntes, documentación y herramientas de IA, pero deben comprender el código que entregan."

🎯 Cómo conectaremos esta clase con el nuevo examen

Esto es importante: **ya tenemos definida la estructura de evaluación que te propondré cuando trabajemos el examen.**

Mi propuesta sería mantener una combinación de conocimientos conceptuales y programación práctica, pero mejorar cómo evaluamos la comprensión.

Competencia	Se enseña en	Cómo la evaluaremos
Fundamentos de Ciencia de Datos y computación	Clases anteriores	Preguntas conceptuales
Variables y tipos de datos	Repaso y clase anterior	Lectura de código
Listas, tuplas y diccionarios	Bloque 2	Ejercicio corto
Ciclos y condicionales	Bloques 1 y 3	Interpretación de código
Funciones y parámetros	Bloques 4 y 6	Construcción de funciones
`return` y procesamiento de listas	Bloques 5, 7 y 8	Ejercicio práctico
Validación y debugging	Bloque 9	Encontrar y corregir errores
Uso responsable de IA	Bloque 9	Explicación y reflexión

Lo que conservaría de tu examen actual

Las preguntas conceptuales sobre Ciencia de Datos, Machine Learning e historia de la computación, porque también forman parte de lo que han aprendido.

Y mantendría **tres problemas prácticos de programación**, con dificultad progresiva.

Lo que cambiaría

En lugar de evaluar únicamente si escribieron una función correcta, evaluaría también si pueden:

- Explicar brevemente su solución.
- Probarla con diferentes entradas.
- Identificar errores.
- Modificar código existente.

Como el examen es en casa y pueden utilizar IA, estas habilidades son más valiosas que pedirles memorizar sintaxis.

Una decisión pedagógica que sí tomaría

No incluiría en el examen conceptos que no hayamos practicado durante la clase, ni siquiera si son fáciles de buscar en ChatGPT.

Eso significa que no metería recursividad, programación orientada a objetos ni estructuras avanzadas.

Y tampoco les pediría algoritmos demasiado elaborados.

Quiero que el examen tenga un nivel de dificultad adecuado para alguien que lleva apenas dos sesiones prácticas de Python.

💜 Mi recomendación final

Hay algo que quiero que observes durante la clase: **cuánto apoyo necesitan para pasar de entender un ejemplo a resolver uno ligeramente diferente**.

Por ejemplo, después de enseñarles una función que filtra calificaciones aprobatorias, ¿pueden modificarla para filtrar reprobatorias?

Si pueden hacerlo, están empezando a comprender la lógica.

Si únicamente pueden reproducir el ejemplo original, necesitan más práctica antes del examen.

Para mí, ese es el mejor indicador de aprendizaje en esta etapa.

Y una última cosa: reservaría los últimos 15 minutos como tiempo flexible. Si ves que todavía están batallando con `return` o con los ciclos dentro de funciones, preferiría reforzar esos conceptos y recortar la actividad de IA.

Porque el objetivo principal del sábado es que entiendan cómo construir funciones. Todo lo demás debe apoyar ese aprendizaje.

Así tendremos una clase coherente y, cuando diseñemos el nuevo examen, podremos evaluar exactamente lo que practicaron. 🐍🚀💜